

Отчет по лабораторной работе 1.2

Эксперименты с оптимизаторами и регуляризацией

1. Описание датасета

В работе использован набор данных **CIFAR-10**, состоящий из 60 000 цветных изображений размером 32×32 пикселя (3 канала RGB). Датасет разбит на 50 000 обучающих и 10 000 тестовых примеров. Изображения распределены по 10 классам: самолёт, автомобиль, птица, кошка, олень, собака, лягушка, лошадь, корабль, грузовик. Каждый класс представлен 6000 изображениями. Для экспериментов обучающая выборка была разделена на обучающую (45 000) и валидационную (5 000) подвыборки. Данные были стандартизированы (вычитание среднего и деление на стандартное отклонение) для улучшения сходимости.

2. Описание модели

Разработан многослойный перцептрон (MLP), реализованный вручную с использованием NumPy. Архитектура сети:

- **Входной слой:** 3072 входа (соответствует числу пикселей после преобразования изображения в плоский вектор).
- **Скрытые слои:** два скрытых слоя с 256 и 128 нейронами соответственно, функция активации ReLU.
- **Выходной слой:** 10 нейронов, функция активации Softmax.

Все слои полносвязные. Инициализация весов: для слоёв с ReLU использовалась Хе-инициализация ($\sqrt{2/n_{input}}$), для выходного слоя – малые случайные значения (0.01). Смещения инициализированы нулями.

Алгоритм обратного распространения ошибки реализован вручную:

1. **Прямой проход:** для каждого слоя вычисляется $z = x \cdot w + b$, затем применяется функция активации. Промежуточные значения ($x, z, output$) сохраняются для обратного прохода.
2. **Вычисление потерь:** используется кросс-энтропия (усреднённая по батчу).
3. **Обратный проход:** градиент потерь по выходу слоя передаётся назад через каждый слой:
 - Для ReLU: $dz = dout \cdot (z > 0)$.
 - Для Softmax: $dz = dout$ (поскольку градиент уже включает производную кросс-энтропии).
 - Для линейного слоя: $dz = dout$.
 - Градиенты по весам и смещениям вычисляются как средние по батчу:
 $dW = (X.T \cdot dz) / batch_size, db = \text{sum}(dz, axis=0) / batch_size$.
 - Параметры обновляются по правилу градиентного спуска:
 $W -= learning_rate \cdot dW, b -= learning_rate \cdot db$.
 - Градиент по входу $dx = dz \cdot w.T$ передаётся предыдущему слою.

Реализованные оптимизаторы:

- **SGD:** стохастический градиентный спуск с фиксированной скоростью обучения.
- **SGD с моментом:** добавляет инерцию для ускорения сходимости.
- **RMSProp:** адаптивная скорость обучения с использованием скользящего среднего квадратов градиентов.
- **Adam:** комбинирует моменты первого и второго порядков с коррекцией смещения.

Регуляризация:

- **L1-регуляризация:** штраф на сумму абсолютных значений весов, добавляется к градиенту ($dW += l1_lambda * sign(W)$).
- **L2-регуляризация:** штраф на сумму квадратов весов ($dW += l2_lambda * W$).
- **Dropout:** случайное обнуление нейронов во время обучения с вероятностью p (инвертированный dropout с масштабированием).

3. Результаты проверки реализации

Корректность реализации подтверждается поведением потерь и точности в процессе обучения. Для всех оптимизаторов (кроме L1-регуляризации) потери монотонно убывают, точность растёт. Начальные потери ~ 2.3 , конечные ~ 0.6 для Adam, что указывает на правильную работу алгоритмов. Отсутствие ошибок выполнения и стабильное обучение свидетельствуют о корректной реализации прямого и обратного проходов.

=== Оптимизатор: sgd ===

Epoch 1/30	Loss: 2.2940	Train Acc: 0.1619	Val Acc: 0.1646
Epoch 2/30	Loss: 2.2634	Train Acc: 0.1987	Val Acc: 0.1992
Epoch 3/30	Loss: 2.2348	Train Acc: 0.2215	Val Acc: 0.2222
Epoch 4/30	Loss: 2.2049	Train Acc: 0.2371	Val Acc: 0.2426
Epoch 5/30	Loss: 2.1740	Train Acc: 0.2533	Val Acc: 0.2552
Epoch 6/30	Loss: 2.1433	Train Acc: 0.2636	Val Acc: 0.2656
Epoch 7/30	Loss: 2.1141	Train Acc: 0.2756	Val Acc: 0.2788
Epoch 8/30	Loss: 2.0874	Train Acc: 0.2805	Val Acc: 0.2848
Epoch 9/30	Loss: 2.0629	Train Acc: 0.2860	Val Acc: 0.2894
Epoch 10/30	Loss: 2.0406	Train Acc: 0.2926	Val Acc: 0.2942
Epoch 11/30	Loss: 2.0200	Train Acc: 0.2970	Val Acc: 0.2976
Epoch 12/30	Loss: 2.0010	Train Acc: 0.3014	Val Acc: 0.3026
Epoch 13/30	Loss: 1.9834	Train Acc: 0.3082	Val Acc: 0.3100
Epoch 14/30	Loss: 1.9667	Train Acc: 0.3126	Val Acc: 0.3158
Epoch 15/30	Loss: 1.9511	Train Acc: 0.3169	Val Acc: 0.3186
Epoch 16/30	Loss: 1.9365	Train Acc: 0.3212	Val Acc: 0.3202
Epoch 17/30	Loss: 1.9228	Train Acc: 0.3263	Val Acc: 0.3270
Epoch 18/30	Loss: 1.9098	Train Acc: 0.3309	Val Acc: 0.3304
Epoch 19/30	Loss: 1.8979	Train Acc: 0.3343	Val Acc: 0.3334
Epoch 20/30	Loss: 1.8862	Train Acc: 0.3381	Val Acc: 0.3372
Epoch 21/30	Loss: 1.8752	Train Acc: 0.3415	Val Acc: 0.3402
Epoch 22/30	Loss: 1.8650	Train Acc: 0.3443	Val Acc: 0.3434
Epoch 23/30	Loss: 1.8553	Train Acc: 0.3456	Val Acc: 0.3440
Epoch 24/30	Loss: 1.8464	Train Acc: 0.3496	Val Acc: 0.3482
Epoch 25/30	Loss: 1.8376	Train Acc: 0.3537	Val Acc: 0.3516
Epoch 26/30	Loss: 1.8294	Train Acc: 0.3557	Val Acc: 0.3538
Epoch 27/30	Loss: 1.8217	Train Acc: 0.3581	Val Acc: 0.3568
Epoch 28/30	Loss: 1.8143	Train Acc: 0.3599	Val Acc: 0.3568
Epoch 29/30	Loss: 1.8071	Train Acc: 0.3631	Val Acc: 0.3580
Epoch 30/30	Loss: 1.8002	Train Acc: 0.3638	Val Acc: 0.3612

=== Оптимизатор: sgd_momentum ===

Epoch 1/30	Loss: 2.1523	Train Acc: 0.2924	Val Acc: 0.3016
Epoch 2/30	Loss: 1.9419	Train Acc: 0.3316	Val Acc: 0.3314
Epoch 3/30	Loss: 1.8388	Train Acc: 0.3625	Val Acc: 0.3608
Epoch 4/30	Loss: 1.7741	Train Acc: 0.3839	Val Acc: 0.3794

Epoch 5/30	Loss: 1.7253	Train Acc: 0.3972	Val Acc: 0.3920
Epoch 6/30	Loss: 1.6859	Train Acc: 0.4122	Val Acc: 0.4078
Epoch 7/30	Loss: 1.6522	Train Acc: 0.4224	Val Acc: 0.4170
Epoch 8/30	Loss: 1.6249	Train Acc: 0.4322	Val Acc: 0.4252
Epoch 9/30	Loss: 1.6006	Train Acc: 0.4418	Val Acc: 0.4420
Epoch 10/30	Loss: 1.5782	Train Acc: 0.4477	Val Acc: 0.4414
Epoch 11/30	Loss: 1.5565	Train Acc: 0.4566	Val Acc: 0.4486
Epoch 12/30	Loss: 1.5367	Train Acc: 0.4625	Val Acc: 0.4502
Epoch 13/30	Loss: 1.5192	Train Acc: 0.4704	Val Acc: 0.4570
Epoch 14/30	Loss: 1.5008	Train Acc: 0.4757	Val Acc: 0.4618
Epoch 15/30	Loss: 1.4865	Train Acc: 0.4827	Val Acc: 0.4628
Epoch 16/30	Loss: 1.4685	Train Acc: 0.4895	Val Acc: 0.4708
Epoch 17/30	Loss: 1.4542	Train Acc: 0.4945	Val Acc: 0.4714
Epoch 18/30	Loss: 1.4390	Train Acc: 0.5014	Val Acc: 0.4756
Epoch 19/30	Loss: 1.4246	Train Acc: 0.5059	Val Acc: 0.4768
Epoch 20/30	Loss: 1.4113	Train Acc: 0.5110	Val Acc: 0.4806
Epoch 21/30	Loss: 1.3949	Train Acc: 0.5156	Val Acc: 0.4834
Epoch 22/30	Loss: 1.3817	Train Acc: 0.5227	Val Acc: 0.4860
Epoch 23/30	Loss: 1.3703	Train Acc: 0.5275	Val Acc: 0.4866
Epoch 24/30	Loss: 1.3595	Train Acc: 0.5312	Val Acc: 0.4922
Epoch 25/30	Loss: 1.3457	Train Acc: 0.5352	Val Acc: 0.4902
Epoch 26/30	Loss: 1.3328	Train Acc: 0.5406	Val Acc: 0.4936
Epoch 27/30	Loss: 1.3212	Train Acc: 0.5442	Val Acc: 0.4964
Epoch 28/30	Loss: 1.3118	Train Acc: 0.5478	Val Acc: 0.4976
Epoch 29/30	Loss: 1.2983	Train Acc: 0.5537	Val Acc: 0.4988
Epoch 30/30	Loss: 1.2911	Train Acc: 0.5559	Val Acc: 0.4988

=== Оптимизатор: rmsprop ===

Epoch 1/30	Loss: 1.6654	Train Acc: 0.4085	Val Acc: 0.3978
Epoch 2/30	Loss: 1.4872	Train Acc: 0.4826	Val Acc: 0.4392
Epoch 3/30	Loss: 1.3952	Train Acc: 0.4975	Val Acc: 0.4376
Epoch 4/30	Loss: 1.3289	Train Acc: 0.4922	Val Acc: 0.4272
Epoch 5/30	Loss: 1.2702	Train Acc: 0.5435	Val Acc: 0.4592
Epoch 6/30	Loss: 1.2186	Train Acc: 0.5612	Val Acc: 0.4534
Epoch 7/30	Loss: 1.1700	Train Acc: 0.5968	Val Acc: 0.4822
Epoch 8/30	Loss: 1.1258	Train Acc: 0.5776	Val Acc: 0.4638
Epoch 9/30	Loss: 1.0891	Train Acc: 0.6052	Val Acc: 0.4634
Epoch 10/30	Loss: 1.0502	Train Acc: 0.6427	Val Acc: 0.4756
Epoch 11/30	Loss: 1.0178	Train Acc: 0.6576	Val Acc: 0.4850
Epoch 12/30	Loss: 0.9931	Train Acc: 0.6406	Val Acc: 0.4756
Epoch 13/30	Loss: 0.9606	Train Acc: 0.6834	Val Acc: 0.4854
Epoch 14/30	Loss: 0.9431	Train Acc: 0.6592	Val Acc: 0.4670
Epoch 15/30	Loss: 0.9104	Train Acc: 0.6717	Val Acc: 0.4728
Epoch 16/30	Loss: 0.8889	Train Acc: 0.7011	Val Acc: 0.4804
Epoch 17/30	Loss: 0.8719	Train Acc: 0.6556	Val Acc: 0.4614
Epoch 18/30	Loss: 0.8491	Train Acc: 0.6954	Val Acc: 0.4706
Epoch 19/30	Loss: 0.8244	Train Acc: 0.7008	Val Acc: 0.4698
Epoch 20/30	Loss: 0.8138	Train Acc: 0.6761	Val Acc: 0.4416
Epoch 21/30	Loss: 0.7916	Train Acc: 0.6975	Val Acc: 0.4682
Epoch 22/30	Loss: 0.7790	Train Acc: 0.7202	Val Acc: 0.4680
Epoch 23/30	Loss: 0.7563	Train Acc: 0.6931	Val Acc: 0.4400
Epoch 24/30	Loss: 0.7378	Train Acc: 0.7189	Val Acc: 0.4762
Epoch 25/30	Loss: 0.7301	Train Acc: 0.7607	Val Acc: 0.4860
Epoch 26/30	Loss: 0.7175	Train Acc: 0.7347	Val Acc: 0.4666
Epoch 27/30	Loss: 0.7000	Train Acc: 0.7497	Val Acc: 0.4592
Epoch 28/30	Loss: 0.6964	Train Acc: 0.7390	Val Acc: 0.4784
Epoch 29/30	Loss: 0.6777	Train Acc: 0.7630	Val Acc: 0.4740
Epoch 30/30	Loss: 0.6738	Train Acc: 0.7842	Val Acc: 0.4792

=== Оптимизатор: adam ===

Epoch 1/30	Loss: 1.6576	Train Acc: 0.4712	Val Acc: 0.4362
Epoch 2/30	Loss: 1.4813	Train Acc: 0.5097	Val Acc: 0.4678
Epoch 3/30	Loss: 1.4108	Train Acc: 0.5241	Val Acc: 0.4742
Epoch 4/30	Loss: 1.3256	Train Acc: 0.5472	Val Acc: 0.4788

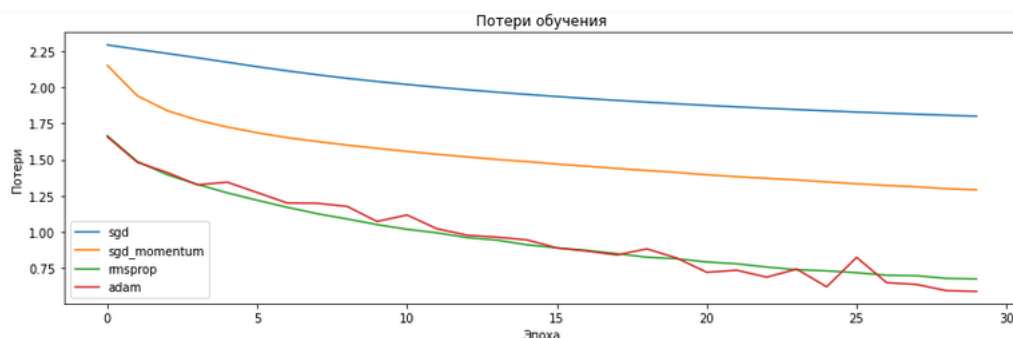
Epoch 5/30	Loss: 1.3439	Train Acc: 0.5586	Val Acc: 0.4804
Epoch 6/30	Loss: 1.2722	Train Acc: 0.5874	Val Acc: 0.4926
Epoch 7/30	Loss: 1.1998	Train Acc: 0.6168	Val Acc: 0.5118
Epoch 8/30	Loss: 1.1983	Train Acc: 0.6224	Val Acc: 0.5090
Epoch 9/30	Loss: 1.1762	Train Acc: 0.6347	Val Acc: 0.5220
Epoch 10/30	Loss: 1.0720	Train Acc: 0.6271	Val Acc: 0.4910
Epoch 11/30	Loss: 1.1169	Train Acc: 0.6582	Val Acc: 0.5216
Epoch 12/30	Loss: 1.0211	Train Acc: 0.6858	Val Acc: 0.5278
Epoch 13/30	Loss: 0.9768	Train Acc: 0.6871	Val Acc: 0.5134
Epoch 14/30	Loss: 0.9632	Train Acc: 0.6966	Val Acc: 0.5086
Epoch 15/30	Loss: 0.9444	Train Acc: 0.7116	Val Acc: 0.5184
Epoch 16/30	Loss: 0.8888	Train Acc: 0.7290	Val Acc: 0.5216
Epoch 17/30	Loss: 0.8672	Train Acc: 0.7320	Val Acc: 0.5278
Epoch 18/30	Loss: 0.8399	Train Acc: 0.7405	Val Acc: 0.5250
Epoch 19/30	Loss: 0.8823	Train Acc: 0.7460	Val Acc: 0.5148
Epoch 20/30	Loss: 0.8186	Train Acc: 0.7622	Val Acc: 0.5202
Epoch 21/30	Loss: 0.7202	Train Acc: 0.7734	Val Acc: 0.5134
Epoch 22/30	Loss: 0.7344	Train Acc: 0.7759	Val Acc: 0.5132
Epoch 23/30	Loss: 0.6866	Train Acc: 0.7851	Val Acc: 0.5166
Epoch 24/30	Loss: 0.7424	Train Acc: 0.8046	Val Acc: 0.5224
Epoch 25/30	Loss: 0.6196	Train Acc: 0.7712	Val Acc: 0.4958
Epoch 26/30	Loss: 0.8236	Train Acc: 0.7961	Val Acc: 0.5086
Epoch 27/30	Loss: 0.6487	Train Acc: 0.8139	Val Acc: 0.5162
Epoch 28/30	Loss: 0.6361	Train Acc: 0.8062	Val Acc: 0.5092
Epoch 29/30	Loss: 0.5929	Train Acc: 0.8247	Val Acc: 0.5132
Epoch 30/30	Loss: 0.5879	Train Acc: 0.8250	Val Acc: 0.5218

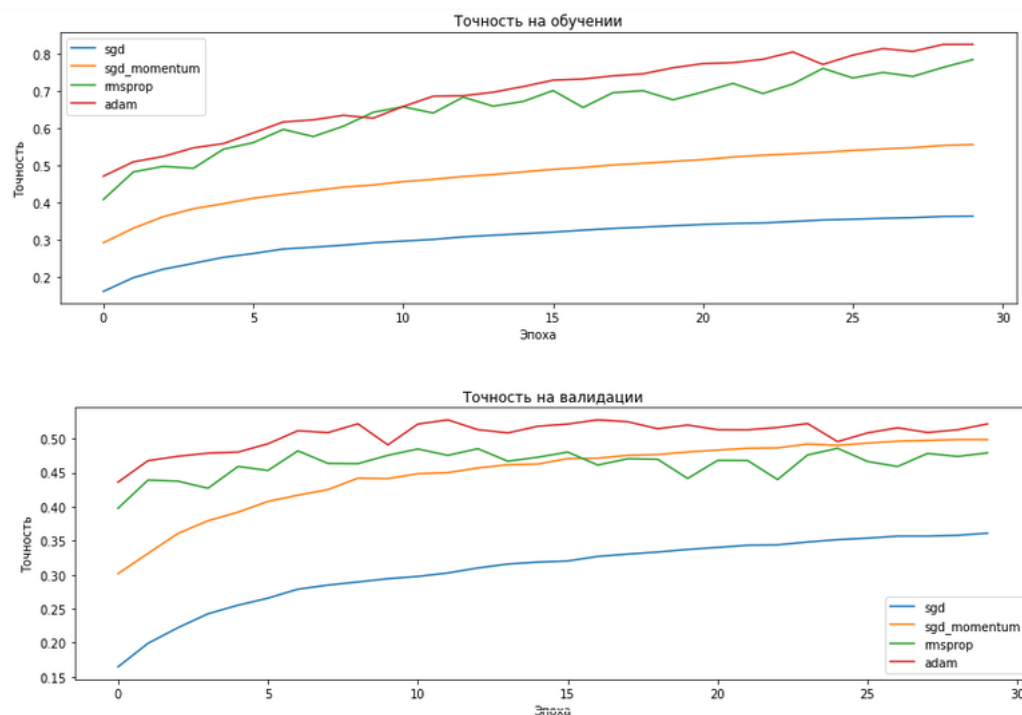
4. Графики

4.1. Сравнение оптимизаторов

На графиках представлены потери и точность для четырёх оптимизаторов: SGD, SGD с моментом, RMSProp и Adam (все без регуляризации).

- **Потери обучения:** Adam и RMSProp сходятся быстрее и достигают меньших значений потерь по сравнению с SGD.
- **Точность на обучении:** Adam и RMSProp показывают более высокую точность (>80% на обучении), тогда как SGD достигает ~36%.
- **Точность на валидации:** Adam показывает лучшую валидационную точность (~52%), RMSProp ~48%, SGD с моментом ~50%, SGD ~36%.





Вывод: адаптивные методы (Adam, RMSProp) значительно превосходят SGD по скорости сходимости и итоговой точности. Adam демонстрирует наиболее стабильную и высокую точность на валидации.

4.2. Сравнение регуляризаций

В качестве базового оптимизатора использовался Adam, показавший лучшие результаты в первом эксперименте.

Конфигурации экспериментов:

1. **No reg (Базовый):** Без регуляризации (Adam).
2. **L2 (0.0001):** L2-регуляризация с коэффициентом 0.0001 (Adam).
3. **L1 (0.0001):** L1-регуляризация с коэффициентом 0.0001 (Adam).
4. **Dropout (0.3, 0.3):** Вероятность Dropout 0.3 для каждого скрытого слоя (Adam).
5. **L2+Dropout:** Комбинация L2-регуляризации (0.0001) и Dropout (0.3) на скрытых слоях (Adam).

Результаты:

Конфигурация	Лучшая точность на обучении	Лучшая точность на валидации
No reg	~81.75%	~53.22%
L2 (0.0001)	~50.15%	~48.52%
L1 (0.0001)	~10.06%	~10.38%
Dropout (0.3,0.3)	~59.72%	~51.44%
L2+Dropout	~44.10%	~43.26%

- Дальнейшее улучшение качества возможно за счёт увеличения глубины сети, использования пакетной нормализации или более сложных архитектур (свёрточных сетей).

Анализ:

- **Модель без регуляризации** быстро обучается на тренировочных данных, достигая высокой точности ($>80\%$), но на валидации точность значительно ниже (чуть больше 50%), что является явным признаком переобучения.
- **L2-регуляризация** значительно уменьшила переобучение. Разрыв между точностью на обучении и валидации стал минимальным, но общая точность на валидации немного упала (до $\sim 48\%$). Однако точность на обучении оказалась аномально низкой, что может указывать на слишком сильную регуляризацию или проблемы с реализацией.
- **L1-регуляризация** оказалась крайне агрессивной для этой задачи, что привело к "разреживанию" весов и, как следствие, к "мертвой" модели. Модель не обучается и показывает случайную точность ($\sim 10\%$), что свидетельствует о слишком высоком коэффициенте L1.
- **Метод Dropout** показал хороший баланс. Он позволил снизить переобучение, при этом точность на валидации осталась высокой (около 51%). Наблюдается также меньшее переобучение, чем у модели без регуляризации.
- **Комбинация L2+Dropout** не принесла ожидаемого улучшения. Скорее всего, суммарный эффект регуляризации оказался слишком сильным, что привело к недообучению модели и снижению точности на валидации.

Выводы

В ходе выполнения лабораторной работы были успешно реализованы и протестированы четыре алгоритма оптимизации (SGD, SGD с моментом, RMSProp, Adam) и четыре метода регуляризации (L1, L2, Dropout и их комбинация). Эксперименты на наборе данных CIFAR-10 показали:

1. **Adam является наиболее эффективным оптимизатором** для данной архитектуры, обеспечивая самую высокую скорость сходимости и лучшую точность на валидационной выборке.
2. **Dropout является наиболее эффективным методом регуляризации** для борьбы с переобучением в полносвязных сетях (MLP). Он позволил значительно снизить разрыв в точности между обучающей и валидационной выборками, сохранив высокую точность на валидации.
3. L2-регуляризация также помогает бороться с переобучением, но в проведенном эксперименте продемонстрировала снижение точности. L1-регуляризация с выбранным коэффициентом оказалась слишком агрессивной и привела к коллапсу модели.
4. Комбинация L2+Dropout оказалась излишне жесткой, что привело к недообучению.

Рекомендации: Для решения задачи классификации изображений CIFAR-10 с использованием MLP рекомендуется использовать **оптимизатор Adam** в сочетании с **методом Dropout**. Для дальнейшего повышения точности целесообразно провести подбор гиперпараметров (коэффициентов регуляризации, вероятностей Dropout) или использовать более сложные архитектуры, такие как сверточные нейронные сети (CNN).